

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-aad97da607c54518e.jsonl`
- SHA-256: `5ad3edc4836e3620890d47cbaf40c0919ef77ae937b4d266ddf0b30ba2a230e4`
- Source modified: `2026-06-21T13:36:54+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:34:48.453Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: How the app is launched + its current packaging surface. READ: pyproject.toml, requirements.txt, backend/main.py (the main() CLI + --server path + how config.json + .env are discovered + where SQLite DBs and output/ live), backend/worker/__main__.py, scripts/ (list + purpose), run_all_tests.py, README.md install/quickstart sections. Answer: EXACTLY what a user must do today to go from nothing to a running server+worker; what `pip install .` provides vs what it does NOT; the console-script entrypoint; how/where config + DB + output dirs are resolved (cwd-relative? configurable?); whether server and worker are one process or two.

2. assistant (2026-06-21T13:34:51.286Z)

I'll analyze the app launch and packaging surface. Let me start by reading the key files.

3. user (2026-06-21T13:34:52.233Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47         "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
48     ]
49
50     # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
```

```

security.edge_auth_enabled=True.
51     # `jwt` is lazy-imported, so the base runtime works without this; install it for
hosted/multi-tenant.
52     auth = ["PyJWT[crypto]>=2.8"]
53
54     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
55     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
56     # fetched over SSH; if you lack access, simply omit this group ? reporting
57     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
58     #   pip install -e ../sports-obsreport --no-build-isolation
59     reporting = [
60         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
61     ]
62
63     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
64     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
65     # express, so they are NOT declarable here. Install them out-of-band instead:
66     #   # CUDA 12.4 box:
67     #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
68     #   # CPU-only box (also what CI uses):
69     #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
70     gpu = []
71
72     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
73     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
74     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
75     # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
76     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
77     storage-s3 = ["boto3>=1.34"]
78     storage-gcs = ["google-cloud-storage>=2.16"]
79
80     [project.scripts]
81     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
82     indexer = "backend.main:main"
83
84     [tool.hatch.metadata]
85     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
86     # reference*); hatchling requires opting in to allow those in metadata.
87     allow-direct-references = true
88
89     [tool.hatch.build.targets.wheel]
90     # `backend` is a namespace package (no __init__.py); list both import roots
91     # explicitly so an editable/wheel install exposes `import backend` and
92     # `import training` regardless of cwd.
93     packages = ["backend", "training"]
94

```

4. user (2026-06-21T13:34:53.120Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).

```

```

13     logging.basicConfig(
14         level=logging.INFO,
15         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16         datefmt='%H:%M:%S'
17     )
18     logger = logging.getLogger(__name__)
19     from fastapi import FastAPI, HTTPException, Request
20     from fastapi.middleware.cors import CORSMiddleware
21     from pydantic import BaseModel
22     from typing import List, Dict, Any, Optional
23
24     from fastapi.staticfiles import StaticFiles
25     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26     from backend.pipeline.validators.base import registry
27     from backend.infrastructure.database import Database
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.stitcher.compiler import VideoCompiler
30     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32     from backend.config import ( # new typed config
33         load_config as load_app_config,
34         AppConfig,
35         StitchingConfig,
36         enforce_startup_checks,
37         StartupCheckError,
38     )
39     from backend.results import Failure # standardized error/result contract
40     from backend.reporting import emit_indexer_report
41     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44
45     app = FastAPI(title="Sports Highlight Indexer API")
46
47     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
48     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
49     # off. M9: the origin list is configurable (security.cors_allow_origins, default [""])
so a hosted
50     # deploy can lock it to its origin. Read at import (a cheap, torch-free config load;
defaults when no
51     # config.json) ? the middleware is fixed at startup.
52     try:
53         _cors_origins = load_app_config().security.cors_allow_origins
54     except Exception:
55         _cors_origins = [""]
56     app.add_middleware(
57         CORSMiddleware,
58         allow_origins=_cors_origins,
59         allow_credentials=False,
60         allow_methods=[""],
61         allow_headers=[""],
62     )
63
64     # Serves static frontend files directly (now under api/ per approved structure)
65     os.makedirs("backend/api/static", exist_ok=True)
66     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
67
68     @app.get("/", response_class=HTMLResponse)
69     def serve_index():
70         index_path = "backend/api/static/index.html"

```

```

71         if os.path.exists(index_path):
72             with open(index_path, "r", encoding="utf-8") as f:
73                 return f.read()
74         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
75
76     # Global variables initialized at startup
77     db_instance: Optional[Database] = None
78     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
79
80     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
81     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
82     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
83     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
84     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
85     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
86     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
87     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
88         JobWaiting,
89         _arm_wake_timer,
90         _drain_due_jobs,
91         _get_executor,
92         _job_to_request,
93         _MAX_DRAIN_WAKE_SEC,
94         _maybe_record_job_cost,
95         _run_claimed_job,
96         _schedule_next_drain_if_waiting,
97     )
98
99     # Legacy thin wrapper for any remaining direct calls during transition.
100    # New code should import load_app_config / AppConfig from backend.config directly.
101    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
102        cfg = load_app_config(config_path)
103        return cfg.model_dump(mode="json")
104
105    # --- API Models ---
106    class ProcessRequest(BaseModel):
107        video_path: str
108        player_pool: Optional[List[str]] = None
109        max_frames: Optional[int] = None
110        segmenter: Optional[str] = None
111
112    class QueryRequest(BaseModel):
113        video_id: str
114        server: Optional[str] = None
115        receiver: Optional[str] = None
116        duration_min: Optional[float] = None
117        event_type: Optional[str] = None
118
119    class CompileRequest(BaseModel):
120        video_id: str
121        segment_ids: List[int]
122        output_filename: str
123
124    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
125        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
126
127        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
128        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
129        and
130        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.

```

```

130     """
131     if segments:
132         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
133     else:
134         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
135
136
137     # --- API Endpoints ---
138     @app.get("/api/config")
139     def get_config():
140         return global_config
141
142     @app.get("/api/health")
143     def health():
144         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
145         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
146         confirm the engine is reachable and which detector is wired. Never raises."""
147         sport = getattr(global_config, "sport", None)
148         indexing = getattr(global_config, "indexing", None)
149         return {
150             "status": "ok",
151             "sport": sport,
152             "default_segmenter": getattr(indexing, "default_segmenter", None),
153         }
154
155     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
156         """The full hash ? validate ? segment ? report pipeline for one video.
157
158         This is the former synchronous /api/process body, unchanged in behavior, now run on
159         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
160         the sync endpoint used to return (UI compatibility); the per-video observability
161         report is still emitted in `finally:` and grafted as response["reports"].
162
163         `progress` is an optional callable(stage: str) used to surface coarse job progress.
164         Raises on unexpected internal errors ? the caller records those as a job failure
165         (after this function has already restored the pre-run segments, see below).
166         """
167         notify = progress or (lambda stage: None)
168
169         notify("hashing")
170         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
171         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
172         # req.video_path (the source URI) is kept for the DB record + report; only the file-
173         # consumers (hash / validators / segmenter) use the resolved local path.
174         # reading
175         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
176         "storage", None))
177         video_id = compute_video_id(local_video)
178         was_reingest = db_instance.get_video(video_id) is not None
179
180         # 1. Run pluggable validators (with-context variant surfaces ffmpegprobe_meta for the
181         # report)
182         notify("validating")
183         logger.info(f"Running validations for {req.video_path}...")
184         val_results, val_context = registry.run_all_with_context(local_video, global_config)
185         failures = [r for r in val_results if not r.passed]
186
187         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
188         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
189         "motion")
190         seg_name = req.segmenter or default_seg
191         segmenter_actual = None
192         segmentation_ran = False

```

```

189         response: Optional[Dict[str, Any]] = None
190     try:
191         if failures:
192             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
193             # status; it no longer destroys the video's prior good segments.
194             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
195         response = {
196             "video_id": video_id,
197             "status": "failed",
198             "message": "Video failed validation checks.",
199             "results": [r.model_dump() for r in val_results],
200         }
201         return response
202
203     # 2. Run segmenter & indexer
204     logger.info("[API] Video passed checks. Segmenting and indexing...")
205     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
206
207     segmenter_cls = segmenter_registry.get(seg_name)
208     if not segmenter_cls:
209         # Submit-time validation already 400s unknown names; this is the defensive
210         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
211         available = list(segmenter_registry.list_available().keys())
212         response = {
213             "video_id": video_id,
214             "status": "failed",
215             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
216             "results": [r.model_dump() for r in val_results],
217             "play_segments": [],
218         }
219         return response
220
221     logger.info(f"[API] Using segmenter: {seg_name}")
222     notify(f"segmenting ({seg_name})")
223     segmenter_actual = seg_name
224     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
225     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
226     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
227     segmenter = segmenter_cls(db_instance, global_config)
228     try:
229         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
230     except Exception:
231         # A raising segmenter must not leave half-written rows: restore the pre-run
232         # state (same destroy-after-confirm contract as the Failure branch), then
let
233         # the job worker record the failure.
234         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
235         raise
236     segmentation_ran = True
237
238     if isinstance(result, Failure):
239         logger.error(f"[API] Segmenter failed: {result.message}")
240         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
241         response = {
242             "video_id": video_id,
243             "status": "failed",
244             "message": f"Segmenter '{seg_name}' failed: {result.message}",
245             "results": [r.model_dump() for r in val_results],

```

```

246         "play_segments": [],
247     }
248     return response
249
250     segments = result.value if hasattr(result, "value") else result
251     notify("finalizing")
252     _finalize_reingest(video_id, segments, play_wm, cand_wm)
253
254     response = {
255         "video_id": video_id,
256         "status": "success",
257         "message": f"Successfully indexed {len(segments)} play segments using
'{{seg_name}}' segmenter.",
258         "results": [r.model_dump() for r in val_results],
259         "play_segments": segments,
260     }
261     return response
262
263     finally:
264         # Always emit the per-video observability report (next to the video, or to
265         # report.output_dir). Never raises. Surface the paths in the response.
266         paths = emit_indexer_report(
267             db=db_instance, video_id=video_id, video_path=req.video_path,
268             config=global_config,
269             val_results=val_results, val_context=val_context,
270             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
271             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
272         )
273         if paths and isinstance(response, dict):
274             response["reports"] = paths
275
276 @app.post("/api/process", status_code=202)
277 def process_video(req: ProcessRequest):
278     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{{job_id}} for state.
279
280     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
281     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
282     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
283     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
284     """
285     allowed_root = getattr(getattr(global_config, "security", None),
286 "allowed_video_root", None)
287     try:
288         validate_input_path(req.video_path, allowed_root=allowed_root)
289         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
290         unsupported
291         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
292         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
293 "storage", None))
294     except ValueError as e:
295         raise HTTPException(status_code=400, detail=str(e)) from e
296     # The local-existence fast-fail applies only to a LOCAL input, and stats the
297     RESOLVED local path
298     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
299     string. A
300     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
301     when the worker
302     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
303     runs for every
304     # input: its null-byte guard always applies, and a configured allowed_video_root
305     (hosted mode)
306     # rejects a non-local URI as out-of-root.
307     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
308         raise HTTPException(status_code=404, detail=f"Video file not found at
'{{req.video_path}}'")

```



```

300     if req.segmenter and not segmenter_registry.get(req.segmenter):
301         available = list(segmenter_registry.list_available().keys())
302         raise HTTPException(
303             status_code=400,
304             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
305         )
306
307     job_id = uuid.uuid4().hex
308     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
309                                 req.player_pool, req.max_frames):
310         raise HTTPException(status_code=500, detail="Failed to persist job record.")
311     _get_executor().submit(_drain_due_jobs)
312     return JSONResponse(
313         status_code=202,
314         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
315     )
316
317
318     @app.get("/api/jobs/{job_id}/cost")
319     def get_job_cost(job_id: str):
320         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
321         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
322         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
323         cost = db_instance.get_job_cost(job_id)
324         if cost is None:
325             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
326         fx = global_config.billing.fx_inr_per_usd
327         usd = cost.get("cost_usd")
328         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
329         cost["fx_inr_per_usd"] = fx
330         return cost
331
332
333     @app.get("/api/videos/{video_id}/cost")
334     def get_video_cost(video_id: str):
335         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
336         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
337         agg = db_instance.get_video_cost(video_id)
338         fx = global_config.billing.fx_inr_per_usd
339         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
340         agg["fx_inr_per_usd"] = fx
341         return agg
342
343
344     @app.get("/api/jobs/{job_id}")
345     def get_job(job_id: str):
346         """Job state: {status, progress, result, reports}. `status` = execution state
347         (queued|running|done|failed); the pipeline verdict is result["status"]."""
348         job = db_instance.get_job(job_id)
349         if not job:
350             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
351         result = job.get("result")
352         return {
353             "job_id": job["id"],
354             "status": job["status"],
355             "progress": job.get("progress"),
356             "video_id": job.get("video_id"),
357             "error": job.get("error"),
358             "result": result,
359             "reports": (result or {}).get("reports"),

```

```

360         "created_at": job.get("created_at"),
361         "started_at": job.get("started_at"),
362         "finished_at": job.get("finished_at"),
363     }
364
365     # --- Chunked upload (B2, PR-2) -----
366     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
367     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
368     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
369     # sequential-part logic, and abort cleanup ? is unchanged.
370     from backend.api import uploads as uploads_api
371
372     app.include_router(uploads_api.router)
373
374
375     # --- Highlight download (B3, PR-2) -----
376
377     @app.get("/api/highlights/{video_id}/{filename}")
378     def download_highlight(video_id: str, filename: str):
379         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
380         (which only writes into output_dir; the browser previously got back an unreachable
381         server-local path)."""
382         try:
383             name = safe_output_filename(filename)
384         except ValueError as e:
385             raise HTTPException(status_code=400, detail=str(e)) from e
386         if not db_instance.get_video(video_id):
387             raise HTTPException(status_code=404, detail="Indexed video record not found.")
388         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
389         path = os.path.join(output_dir, name)
390         try:
391             path = resolve_under_root(path, output_dir)
392         except ValueError as e:
393             raise HTTPException(status_code=400, detail=str(e)) from e
394         if not os.path.isfile(path):
395             raise HTTPException(status_code=404,
396                               detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
397         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
398         return FileResponse(path, media_type=media, filename=name)
399
400
401     @app.post("/api/query")
402     def query_play_segments(req: QueryRequest):
403         filters = {
404             "server": req.server,
405             "receiver": req.receiver,
406             "duration_min": req.duration_min,
407             "event_type": req.event_type
408         }
409         segments = db_instance.query_play_segments(req.video_id, filters)
410         return {"play_segments": segments}
411
412     @app.post("/api/compile")
413     def compile_highlights(req: CompileRequest):
414         video = db_instance.get_video(req.video_id)
415         if not video:
416             raise HTTPException(status_code=404, detail="Indexed video record not found.")
417
418         # Retrieve matching play segments from db
419         all_segments = db_instance.query_play_segments(req.video_id, {})

```

```

420     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
421
422     if not matched_segments:
423         raise HTTPException(status_code=400, detail="No matching play segments found to
424 stitch.")
425
426     # Reject path traversal / absolute output names (os.path.join would otherwise let an
427 # absolute or '..'-laden output_filename write anywhere on disk).
428     try:
429         out_name = safe_output_filename(req.output_filename)
430     except ValueError as e:
431         raise HTTPException(status_code=400, detail=str(e)) from e
432
433     # The configured shot-count floor (stitching.min_shot_count) applies on the API
434 # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
435 shot_count
436 # metadata are exempt: the floor filters known counts only, so explicitly
437 # requested manual/legacy segments can't silently vanish under the default floor.
438 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
439 kept = []
440 for s in matched_segments:
441     meta = s.get("metadata") or {}
442     sc = meta.get("shot_count") if isinstance(meta, dict) else None
443     try:
444         if sc is not None and int(sc) < stitching.min_shot_count:
445             continue
446     except (TypeError, ValueError):
447         pass # unparseable count -> treat as unknown, keep
448     kept.append(s)
449 if len(kept) < len(matched_segments):
450     logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
451 dropped "
452 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
453 segments below the floor.")
454 if not kept:
455     raise HTTPException(status_code=400,
456 detail=f"All {len(matched_segments)} matching segments fall
457 below "
458 f"stitching.min_shot_count={stitching.min_shot_count}.")
459
460 # Extract start/end time pairs
461 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
462
463 # Run compiler with the configured stitching paddings + optional branding watermark
464 output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
465 or "output"
466 compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
467 end_padding=stitching.end_padding,
468 watermark=stitching.watermark)
469 try:
470     out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
471     # `filepath` is server-local (not reachable by the browser); also return the
472 ready-to-use
473     # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
474 §4.3).
475     download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
476     return {"status": "success", "filepath": out_path, "download_url": download_url}
477 except Exception as e:
478     raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
479 {str(e)}") from e
480
481 # --- CLI Debug Utility & Orchestration ---
482 def run_cli_diagnose_clip(video_path: str, timestamp: float):
483     """CLI debugging utility to examine segment properties around a timestamp."""

```

```

474     print("=" * 60)
475     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
476     print("=" * 60)
477
478     cfg = load_config() # legacy wrapper still works, returns dict for now
479
480     # 1. Validation check diagnostics
481     print("[DIAGNOSTIC] Running validation framework...")
482     results = registry.run_all(video_path, cfg)
483     for r in results:
484         status_symbol = "[PASS]" if r.passed else "[FAIL]"
485         print(f"    {status_symbol} {r.validator_name}: {r.message}")
486         if r.details:
487             print(f"        Details: {r.details}")
488
489     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
490     print("-" * 60)
491     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
492     import cv2
493     cap = cv2.VideoCapture(video_path)
494     if not cap.isOpened():
495         print("[FAIL] OpenCV could not open video.")
496         return
497
498     fps = cap.get(cv2.CAP_PROP_FPS)
499     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
500
501     start_frame = max(0, int((timestamp - 3.0) * fps))
502     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
503
504     # Measure frame-by-frame changes in sample region
505     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
506     prev_gray = None
507     motions = []
508
509     for _ in range(start_frame, end_frame):
510         ret, frame = cap.read()
511         if not ret:
512             break
513         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
514         gray = cv2.GaussianBlur(gray, (21, 21), 0)
515
516         if prev_gray is not None:
517             diff = cv2.absdiff(prev_gray, gray)
518             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
519             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
520             motions.append(motion_ratio)
521             prev_gray = gray
522
523     cap.release()
524
525     if motions:
526         avg_motion = sum(motions) / len(motions)
527         # Support both legacy dict (from wrapper) and future direct model
528         if hasattr(cfg, "indexing"):
529             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
530         else:
531             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
532         print(f"    Sample Frame Count: {len(motions)}")
533         print(f"    Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
534         if avg_motion > threshold:
535             print("    Result: [ACTIVE PLAY] High motion detected. Segment classifies as

```

```

active play.")
536         else:
537             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
538         else:
539             print(" [ERROR] No visual frames were extracted in the sample range.")
540
541     print("=" * 60)
542
543
544     def _enforce_startup_or_exit(config) -> None:
545         """M5: run the per-profile startup checks for the server/CLI path and
`sys.exit(1)` on a
546         FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
any work.
547         A strict no-op when no `deployment.profile` is selected (today's behaviour). No
GPU probe
548         here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
549         so the fail-fast wiring + the default-OFF no-op are unit-testable."""
550         try:
551             enforce_startup_checks(config, logger=logger)
552         except StartupCheckError as e:
553             logger.error("[STARTUP] refusing to start: %s", e)
554             sys.exit(1)
555
556
557     def main():
558         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
559         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
560         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
561         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
562         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
563         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
564         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
565         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
566
567         available_segmenters = list(segmenter_registry.list_available().keys())
568         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
569         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
570         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
571         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
572
573         args = parser.parse_args()
574
575         if args.setup:
576             # Invoke the diagnostics/bootstrap script (renamed from the old root
577             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
578             # build system). Resolve its path from this file so `indexer --setup`
579             # works regardless of the caller's cwd.
580             import subprocess
581             doctor_path = os.path.join(
582                 os.path.dirname(os.path.abspath(__file__)),
583                 "scripts", "doctor.py",

```

```

584         )
585         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
586         subprocess.run([sys.executable, doctor_path])
587         return
588
589     if args.trim_start is not None and args.trim_end is not None:
590         if not args.video_path:
591             print("[ERROR] Please provide --video-path to extract a segment.")
592             return
593
594         # Determine output path
595         out_filename = args.trim_output or "trimmed_segment.mp4"
596         if os.path.isabs(out_filename):
597             out_path = out_filename
598             out_dir = os.path.dirname(out_path)
599             out_name = os.path.basename(out_path)
600         else:
601             out_dir = args.output_dir
602             out_path = os.path.join(out_dir, out_filename)
603             out_name = out_filename
604
605         os.makedirs(out_dir, exist_ok=True)
606         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
607
608         # global_config isn't initialized this early; load the typed config so the trim
609         # clip carries the same configured stitching paddings + watermark as a real
610         # highlight (watermark default-on).
611         stitching = load_app_config().stitching
612         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
613                                 end_padding=stitching.end_padding,
614                                 watermark=stitching.watermark)
615         try:
616             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
617 args.trim_end)], out_name)
618             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
619         except Exception as e:
620             print(f"[ERROR] Failed to compile trimmed segment: {e}")
621         return
622
623     if args.debug_clip is not None:
624         if not args.video_path:
625             print("[ERROR] Please provide --video-path to debug a specific clip.")
626             return
627         run_cli_diagnose_clip(args.video_path, args.debug_clip)
628         return
629
630     # Initialize Global Config and DB
631     global global_config, db_instance
632     global_config = load_app_config() # returns typed AppConfig
633     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
634     profile
635
636     # The CLI --output-dir overrides the configured output directory. It now lives
637     # on the typed model (OutputConfig.output_dir), so every consumer ? including
638     # /api/compile ? reads the same value instead of a write-only runtime hack.
639     global_config.output.output_dir = args.output_dir
640     os.makedirs(global_config.output.output_dir, exist_ok=True)
641
642     db_path = os.path.join(global_config.output.output_dir,
643 global_config.output.db_name)
644     db_instance = Database(db_path)
645
646     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
647     # the executor is in-process. Mark them failed so pollers see a terminal state.

```

```

644     swept = db_instance.fail_stale_jobs()
645     if swept:
646         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
647
648     # CLI processing mode (no web UI needed)
649     if args.video_path and not args.server:
650         print(f"[CLI] Processing video file: {args.video_path}")
651         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
        # fetched to scratch.
652         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
653         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
654         storage_cfg = getattr(global_config, "storage", None)
655         try:
656             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
657         except ValueError as e:
658             print(f"[FAIL] {e}")
659             return
660         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
661             print(f"[FAIL] Video file not found: {args.video_path}")
662             return
663         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
664
665         video_id = compute_video_id(local_video)
666         was_reingest = db_instance.get_video(video_id) is not None
667
668         # Validate (with-context variant surfaces ffprobe_meta for the report)
669         print("[CLI] Validating...")
670         val_results, val_context = registry.run_all_with_context(local_video,
global_config)
671         failures = [r for r in val_results if not r.passed]
672
673         # Save video status
674         status = "failed" if failures else "processed"
675         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
676
677         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
678         segmenter_actual = None
679         segmentation_ran = False
680         try:
681             print("\n" + "="*40)
682             print("VALIDATION SUMMARY")
683             print("="*40)
684             for r in val_results:
685                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
686                 print(f"{status_symbol} {r.validator_name}: {r.message}")
687             print("="*40 + "\n")
688
689             if failures:
690                 print("[FAIL] Video failed checks. Indexing halted.")
691                 return
692
693             # Index
694             segmenter_cls = segmenter_registry.get(seg_name)
695             if not segmenter_cls:
696                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
697                 return
698
699             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
700             segmenter_actual = seg_name
701             play_wm, cand_wm = db_instance.segment_watermarks(video_id)

```

```

702         segmenter = segmenter_cls(db_instance, global_config)
703         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
704         segmentation_ran = True
705
706         if isinstance(result, Failure):
707             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
708             print("[FAIL] Indexing halted for video.")
709             segments = []
710         else:
711             segments = result.value if hasattr(result, "value") else result
712             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
713             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
714             _finalize_reingest(video_id, segments, play_wm, cand_wm)
715         finally:
716             # Always emit the per-video observability report. Never raises.
717             paths = emit_indexer_report(
718                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
719                 val_results=val_results, val_context=val_context,
720                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
721                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
722             )
723             if paths:
724                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
725                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
726                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
727             return
728
729         # Start Local Server Mode
730         if args.server or not args.video_path:
731             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
732             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
733             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
734             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
735             _get_executor().submit(_drain_due_jobs)
736             uvicorn.run(app, host="127.0.0.1", port=args.port)
737
738     if __name__ == "__main__":
739         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
740         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
741         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
742         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
743         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
744         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
745         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
746         import backend.main as _bm
747
748         _bm.main()
749

```

5. assistant (2026-06-21T13:34:55.618Z)

Let me read the remaining key files in parallel.

6. user (2026-06-21T13:34:57.236Z)

```
1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v       # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18       package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
26     import sys
27
28
29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):
33             return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print("Full test suite: " + " ".join(cmd))
47         print(f"obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print("RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60
```

7. user (2026-06-21T13:34:57.380Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import sys
28     import tempfile
29     from typing import Any, List
30
31     from backend import storage
32     from backend.compute import ComputeJobSpec, get_compute_backend
33     from backend.config import (
34         StartupCheckError,
35         enforce_startup_checks,
36         load_config,
37         probe_cuda_available,
38     )
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.validators.base import registry as validator_registry
42     from backend.reporting import emit_indexer_report
43     from backend.results import Failure
44     from backend.utils.hashing import compute_video_id
45
46     logger = logging.getLogger("backend.worker")
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
```

```

59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")
62             if not sep:
63                 raise ValueError(f"--set expects k=v, got {kv!r}")
64             parts = key.split(".")
65             obj = config
66             for p in parts[:-1]:
67                 obj = getattr(obj, p)
68                 setattr(obj, parts[-1], _coerce(val))
69
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks

```

```

114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                     result.video_id, result.returncode, result.outputs_uri)
137         return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                             status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
144         try:
145             marker = {"video_id": video_id, "returncode": returncode,
146                      "segment_count": segment_count, "status": status}
147             local = os.path.join(scratch, "_done.json")
148             with open(local, "w", encoding="utf-8") as f:
149                 json.dump(marker, f)
150             storage.put(local, done_uri, config=storage_cfg)
151             logger.info("[worker] wrote completion marker -> %s", done_uri)
152         except Exception as e: # never fail a good run because the marker push hiccuped
153             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154
155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
163         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
164         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
165         # dispatcher forces compute_target=local_gpu), so it never recursively re-

```

```

dispatches.
166     compute = get_compute_backend(config)
167     if compute is not None:
168         return _dispatch_remote(compute, args)
169
170     storage_cfg = config.storage.model_dump()
171
172     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
173     os.makedirs(scratch, exist_ok=True)
174     config.output.output_dir = scratch
175
176     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
177     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
178     print(f"[worker] pulled {args.video_uri} -> {local_video}")
179
180     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
181     video_id = compute_video_id(local_video)
182
183     # 3. VALIDATE (same registry as the main CLI).
184     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
185     failures = [r for r in val_results if not r.passed]
186     db = Database(os.path.join(scratch, config.output.db_name))
187     db.add_video(video_id, local_video, "failed" if failures else "processed",
188                 [r.model_dump() for r in val_results])
189
190     segments: List[dict] = []
191     segmenter_actual = None
192     segmentation_ran = False
193     status = "failed"
194     try:
195         if failures:
196             print("[worker] validation FAILED: "
197                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
198             return 2
199         seg_name = args.segmenter or config.indexing.default_segmenter
200         segmenter_cls = segmenter_registry.get(seg_name)
201         if not segmenter_cls:
202             print(f"[worker] unknown segmenter: {seg_name!r} "
203                   f"(have {list(segmenter_registry.list_available())})")
204             return 2
205         segmenter_actual = seg_name
206         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
207         segmentation_ran = True
208         if isinstance(result, Failure):
209             print(f"[worker] segmenter FAILED: {result.message}")
210             return 3
211         segments = result.value if hasattr(result, "value") else result
212         status = "success"
213         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
214         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
215         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
216         if not segments:
217             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
218                             or getattr(config.indexing, "detector_impl", None) or
"auto")
219             logger.warning(
220                 "infer produced 0 segments for video_id=%s (segmenter=%s,

```

```

detector_impl=%s). "
221             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
222             "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
223             "the WASB env produced no detections, or the input resolution differs
from the tuned "
224             "proxy resolution. Re-run with -v for the native command + frame
counts.",
225             video_id, segmenter_actual, detector_impl,
226         )
227     finally:
228         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229         emit_indexer_report(
230             db=db, video_id=video_id, video_path=local_video, config=config,
231             val_results=val_results, val_context=val_context,
232             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233             was_reingest=False, segmentation_ran=segmentation_ran,
234         )
235
236     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
237     out_local = os.path.join(scratch, "outputs", video_id)
238     os.makedirs(out_local, exist_ok=True)
239     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
240     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
241
242     out_prefix = args.out_uri.rstrip("/")
243     pushed = []
244     for fn in sorted(os.listdir(out_local)):
245         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
246         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
247         pushed.append(uri)
248
249     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
250     for u in pushed:
251         print("    ", u)
252
253     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
254     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
255     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
256     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
257     if getattr(args, "done_uri", None):
258         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
259     return 0
260
261
262     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
263     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
264     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
265
266
267     def _probe_video_fps_width(path: str):
268         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
269
270         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a

```

WRONG

```
271     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
272     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
273     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
274     probe with ffmpeg (same tool as ``get_video_duration``) and return None so the
caller SKIPS
275     rather than guesses.
276     """
277     import json
278     import subprocess
279     try:
280         out = subprocess.check_output(
281             ["ffmpeg", "-v", "error", "-select_streams", "v:0",
282              "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
283             stderr=subprocess.DEVNULL).decode()
284         st = (json.loads(out).get("streams") or [{}])[0]
285         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
286         fps = float(num) / float(den or "1")
287         width = float(st.get("width") or 0)
288         if fps > 0 and width > 0:
289             return fps, width
290     except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
291         pass
292     return None
293
294
295 def _resolve_train_detector(args: argparse.Namespace, config: Any):
296     """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
297     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
298     stub=CPU mock). Returns the runner, or prints an error + returns None.
299
300     Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
301     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
302     no shuttle detector and is rejected.
303     """
304     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
305
306     seg_cls = segmenter_registry.get(args.segmenter)
307     if not seg_cls:
308         print(f"[worker] unknown segmenter: {args.segmenter!r} "
309               f"(have {list(segmenter_registry.list_available())})")
310         return None
311     seg = seg_cls(None, config)
312     if not isinstance(seg, TrajectoryHybridSegmenter):
313         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
314               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
315         return None
316     try:
317         runner, _ = seg._resolve_runner(config.get("indexing", {}))
318     except NotImplementedError as e:
319         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
320         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
321         print(f"[worker] detector not available: {e}")
322         return None
323     ok, msg = runner.healthcheck()
324     if not ok:
```

```

325         print(f"[worker] detector environment not ready: {msg}")
326         return None
327     print(f"[worker] detector ready ({args.segmenter}): {msg}")
328     return runner
329
330
331     def cmd_train(args: argparse.Namespace) -> int:
332         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
333         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
334
335         Two trajectory sources (the train pipeline + harness are identical either way):
336         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
337         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
338         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
339         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
340         is the M3.5 "first real training" path; only the trajectory source flips.
341         """
342         import subprocess
343
344         config = load_config(args.config) if args.config else load_config()
345         _apply_overrides(config, args.set)
346         if not _run_startup_checks(config):
347             return 2
348         storage_cfg = config.storage.model_dump()
349
350         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
351         labels_dir = os.path.join(scratch, "labels")
352         traj_dir = os.path.join(scratch, "trajectories")
353         videos_dir = os.path.join(scratch, "videos")
354         os.makedirs(labels_dir, exist_ok=True)
355         os.makedirs(traj_dir, exist_ok=True)
356
357         corpus = args.corpus_uri.rstrip("/")
358         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
359                             if u.lower().endswith(".csv"))
360         if len(label_uris) < 2:
361             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
362                   f"under {corpus}/labels/")
363             return 2
364
365         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
366         runner = None
367         video_by_stem: dict = {}
368         if args.trajectory_source == "detector":
369             os.makedirs(videos_dir, exist_ok=True)
370             runner = _resolve_train_detector(args, config)
371             if runner is None:
372                 return 2
373             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
374                 vname = storage.parse_uri(vuri).name
375                 if not vname.lower().endswith(_VIDEO_EXTS):
376                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
377                 video_by_stem[os.path.splitext(vname)[0]] = vuri
378
379         print(f"[worker] trajectory source: {args.trajectory_source}")
380         specs: List[dict] = []

```



```

381         for uri in label_uris:
382             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
383             name = fname.replace(".rallies.csv", "").replace(".csv", "")
384             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
385             if args.trajectory_source == "detector":
386                 vuri = video_by_stem.get(name)
387                 if not vuri:
388                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
389                     continue
390                     local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
391                                     config=storage_cfg)
392                     video_id = compute_video_id(local_video)
393                     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
394                     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
395                     meta = _probe_video_fps_width(local_video)
396                     if meta is None:
397                         print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
398                         continue
399                     fps, frame_width = meta
400                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
401                     if not traj:
402                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
403                         continue
404                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
405                                "fps": fps, "frame_width": frame_width})
406             else:
407                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
408                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
409                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
410                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
411                              "fps": args.fps, "frame_width": args.frame_width})
412
413             if len(specs) < 2:
414                 print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
415                 return 2
416             manifest_path = os.path.join(scratch, "manifest.json")
417             with open(manifest_path, "w", encoding="utf-8") as f:
418                 json.dump(specs, f, indent=2)
419             src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
420             print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
421
422             out_dir = os.path.join(scratch, "artifacts")
423             cmd = [sys.executable, "-m", "training.gen0.harness",
424                   "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
425             if args.floor:
426                 floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
427                               if "://" in args.floor else args.floor)
428                 cmd += ["--floor", floor_local]
429             print("[worker] training:", " ".join(cmd))
430             rc = subprocess.run(cmd).returncode
431             if rc != 0:
432                 print(f"[worker] gen0 harness failed (rc={rc})")

```

```

433         return rc
434
435     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
436     bundle = os.path.join(out_dir, args.version)
437     out_prefix = args.out_uri.rstrip("/")
438     pushed = []
439     for fn in sorted(os.listdir(bundle)):
440         uri = f"{out_prefix}/weights/{args.version}/{fn}"
441         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
442         pushed.append(uri)
443     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
444     for u in pushed:
445         print("    ", u)
446     return 0
447
448
449     def build_parser() -> argparse.ArgumentParser:
450         p = argparse.ArgumentParser(prog="python -m backend.worker",
451                                     description="Storage-aware worker: pull ? run pipeline ?
452 push.")
453         p.add_argument("-v", "--verbose", action="store_true",
454                       help="DEBUG logging (the native detector command, frame counts, per-
455 stage detail)")
456         sub = p.add_subparsers(dest="cmd", required=True)
457
458         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
459         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
460 gcs://")
461         pi.add_argument("--out-uri", required=True, help="output prefix
462 (outputs/<video_id>/? is appended)")
463         pi.add_argument("--segmenter", default=None, help="default:
464 indexing.default_segmenter")
465         pi.add_argument("--config", default=None, help="config.json path (default:
466 ./config.json)")
467         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
468 dir)")
469         pi.add_argument("--max-frames", type=int, default=None)
470         pi.add_argument("--done-uri", default=None,
471                       help="M6: storage URI for a completion MARKER (video_id + rc)
472 written on a "
473
474                       "successful run, so a remote (Cloud Run) dispatcher's bucket-
475 poll terminates. "
476
477                       "Default-OFF: omit it for the normal local run.")
478         pi.add_argument("--set", action="append", default=[], metavar="k=v",
479                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
480         pi.set_defaults(func=cmd_infer)
481
482         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
483 trajectories -> harness)")
484         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
485 videos/ for "
486
487                       "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
488         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
489 is appended)")
490         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
491         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
492                       default="synth",
493                       help="synth = CPU-mock label-consistent trajectories (default, no
494 GPU); "
495
496                       "detector = REAL shuttle trajectories via the resolved
497 DetectorRunner "
498
499                       "(needs videos/ in the corpus; honours detector_impl).")
500         pt.add_argument("--segmenter", default="wasb_hybrid",
501                       help="trajectory hybrid whose detector is used for --trajectory-
502 source detector")

```

```

482     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
483     pt.add_argument("--config", default=None)
484     pt.add_argument("--scratch", default=None)
485     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
486     pt.add_argument("--frame-width", type=float, default=1280.0,
487                     help="synth trajectory frame width (detector probes the video)")
488     pt.add_argument("--set", action="append", default=[], metavar="k=v")
489     pt.set_defaults(func=cmd_train)
490     return p
491
492
493     def _setup_logging(verbose: bool) -> None:
494         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
495         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
496         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
497         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
498         entry), not at import, so importing the module never reconfigures a caller's
logging."""
499         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
500         # configured logging at import), which would leave the level at WARNING and re-
suppress the
501         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
502         logging.basicConfig(
503             level=logging.DEBUG if verbose else logging.INFO,
504             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
505             datefmt="%H:%M:%S",
506             force=True,
507         )
508
509
510     def main(argv=None) -> int:
511         args = build_parser().parse_args(argv)
512         _setup_logging(getattr(args, "verbose", False))
513         return args.func(args)
514
515
516     if __name__ == "__main__":
517         sys.exit(main())
518

```

8. user (2026-06-21T13:34:58.398Z)

```

1     # Badminton Highlight & Rally Indexer
2
3     A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5     The pipeline is built around three pluggable registries ? validators,
segmenters, and AI providers ? plus a sport profile registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7     > Where this is heading: the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection offline from a commercially-licensed,
annotated corpus. See [docs/ROADMAP.md](docs/ROADMAP.md) for the four-phase plan and
[docs/archives/decisions/DECISIONS.md](docs/archives/decisions/DECISIONS.md) (historical

```

```

ADRs) for the rationale.
8
9     ---
10
11     ## ? Key Features
12
13     * **Pluggable Video Validation Framework**: Easily extendable base-class architecture
that automatically runs checks on:
14     * *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
15     * *PTS Timeline Continuity*: Scans keyframe timestamps for timeline gaps, jumps,
or editing splices to detect tampering or splices.
16     * *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17     * *Blurriness Check*: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18     * *Relevance (Is it Badminton?)*: Uses high-performance HSV color court matching
to identify court layouts.
19     * **Six Pluggable Rally Segmenters** (`backend/pipeline/segmenters/` ? `motion`,
`gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`). The three core
ones:
20     * `yolo_hybrid` ? two-stage pipeline. Stage 1 runs **torchvision person
detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-licensed)
to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands
each candidate (with a small padding) to the AI provider for high-precision semantic boundaries
and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key
remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive
components ? see [docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and
ADR-005.)*
21     * `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22     * `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23     * **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24     * **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25     * **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26     * **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27     * **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28     * **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel
(**on by default** ? `"Generated by Khelsutra.guru"`, fully configurable, with an off switch).
29     * **Detailed CLI Debugging Tools**: --debug-clip <t> inspects frame metrics,
validator scores, and motion values around a timestamp. --trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31     ---
32
33     ## ?? Setup & Diagnostics
34
35     ### 1. External System Requirements
36     The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38     * **On Windows (PowerShell as Admin)**:

```

```

39     ```powershell
40     winget install Gyan.FFMpeg
41     ```
42     *Restart your terminal afterwards to register the path changes.*
43
44     ### 2. Install the package
45
46     The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling
47     build backend). Install it editable:
48
49     ```bash
50     # Runtime deps only:
51     pip install -e .
52     # Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
53     pip install -e .[dev]
54     ```
55
56     This registers the `indexer` console entry point (so `indexer --server`,
57     `indexer --help`, etc. work) and puts the `backend` and `training` packages on
58     your path.
59
60     > **PyTorch is installed separately.** `torch`/`torchvision` are intentionally
61     > *not* in `pyproject.toml`: their CPU/CUDA wheels need an out-of-band wheel index
62     > that PEP 621 cannot express. Install them yourself, matching your hardware:
63     > ```bash
64     > # NVIDIA GPU (CUDA 12.4):
65     > pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url
66     https://download.pytorch.org/whl/cu124
67     > # CPU only:
68     > pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
69     > ```
70
71     ### 3. Diagnostics Checker (`scripts/doctor.py`)
72     Run the diagnostics checker to verify your system dependencies, initialize a
73     default config, and (optionally) bootstrap the Python packages:
74
75     ```bash
76     python scripts/doctor.py
77     # or, via the CLI:
78     indexer --setup
79     ```
80
81     This script will:
82     * Validate `ffmpeg` and `ffprobe` are present in your PATH.
83     * Initialize a default `config.json` (if one does not exist) with thresholds for blur,
84     court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
85     * Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
86     PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
87     * Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
88     `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
89     `supervision`.
90
91     > The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster
92     R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
93     nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
94     `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
95     `retinanet_resnet50_fpn`.)
96
97     ---
98
99     ## ? Running the Application
100
101     ### 1. Launching the Local Web App Dashboard
102     To start the FastAPI server and open the web dashboard:
103
104     ```bash
105     python -m backend.main --server --port 8000
106     ```

```

```

95     Open your browser and navigate to **[http://localhost:8000](http://localhost:8000)**.
96     *Simply input your local raw video file path in the sidebar to process, validate, index,
and compile highlights instantly.*
97
98     ### 2. Running in CLI Processing Mode
99     To process and index a video completely via terminal commands:
100     ```bash
101     python -m backend.main --video-path "C:/path/to/my_video.mp4"
102     ```
103     This outputs a validation summary and indexes all rallies directly into the local SQLite
database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The
shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
104
105     To explicitly pick a segmenter for one run:
106     ```bash
107     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
108     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
109     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
110     ```
111
112     ### 3. Rich CLI Debugging (`--debug-clip`)
113     To diagnose exactly why a video segment was registered as active/dead play, or inspect
detailed focus/continuity metrics at a particular time in seconds:
114     ```bash
115     python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
116     ```
117     *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
window around the timestamp.*
118
119     ### 4. Extracting a Trimmed Segment (`--trim-start` / `--trim-end`) ? also the quickest
watermark preview
120     Extract a slice of a recording to disk (re-encoded; and, with the default-on branding
watermark, stamped ***Generated by Khelsutra.guru*** in the corner):
121     ```bash
122     python -m backend.main --video-path "C:/path/to/my_video.mp4" \
123     --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
124     ```
125     Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path. This is the fastest way to *see* the watermark on a real clip ? set
`stitching.watermark.enabled: false` in `config.json` to turn it off.
126
127     > To preview the exact overlay on any file without the pipeline:
128     > ```bash
129     > ffmpeg -i in.mp4 -vf "drawtext=text='Generated by Khelsutra.guru':font='DejaVu Sans':fontcolor=white@0.85:fontsize=h/28:x=w-tw-24:y=h-th-24:box=1:boxcolor=black@0.4:boxborderw=10"
-c:a copy out.mp4
130     > ```
131
132     ### 5. End-to-End "Process + Stitch" Script
133     `scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes
a hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
settings against a fixed large recording without re-running expensive segmentation.
134
135     ### 6. Cache Hygiene & Disk Cleanup
136     The trajectory detectors (`wasb_hybrid` / `tracknet_hybrid`) stage a copy of each video
into WSL and decode every frame to PNG ? easily **tens of GBs per 4K video**. To keep disk usage
bounded:
137
138     * **Automatic self-clean:** after a *successful* run the frame cache + staged copy are
deleted (`indexing.{wasb,tracknet}.keep_frames`, default `false`). On failure they're kept so a
re-run resumes from the checkpointed detector cache.

```

```

139      *   **Manual sweep (dry-run-first):** `tools/cleanup_caches.py` reports and (with
`--apply`) reclaims what failed/old runs left behind, across both the WSL stage dir and
`output/`:
140      ```bash
141      python -m tools.cleanup_caches                # report what's reclaimable (safe;
deletes nothing)
142      python -m tools.cleanup_caches --apply          # delete the regenerable caches
143      python -m tools.cleanup_caches --keep-video GX010125 # protect one video's cache
(id or filename stem)
144      python -m tools.cleanup_caches --older-than 7    # only purge entries older than 7
days
145      ```
146      It treats frame caches, staged/proxy videos, and handoff/temp dirs as
**regenerable** (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels.
Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line
nudge (used by the optional SessionStart hook).
147
148      ---
149
150      ## ?? Threshold Tuning (`config.json`)
151      You can adjust the parameters of your validators, segmenters, AI provider, and stitcher
by editing `config.json` in the root folder. The default config emitted by `scripts/doctor.py`:
152
153      ```json
154      {
155          "sport": "badminton",
156          "ai_provider": "gemini",
157          "ai_model": "gemini-flash-latest",
158          "validation": {
159              "min_resolution_width": 1280,
160              "min_resolution_height": 720,
161              "min_fps": 29.9,
162              "min_blur_threshold": 20.0,
163              "max_scene_cuts_per_minute": 0.5,
164              "relevance": {
165                  "court_green_lower": [35, 40, 40],
166                  "court_green_upper": [85, 255, 255],
167                  "court_pixels_min_ratio": 0.05
168              }
169          },
170          "indexing": {
171              "default_segmenter": "gemini",
172              "use_gpu": true,
173              "motion_threshold": 0.08,
174              "min_segment_duration": 3.0,
175              "dead_time_merge_gap": 2.0,
176              "chunk_duration": 90,
177              "chunk_overlap": 10,
178              "yolo_velocity_threshold": 0.15,
179              "yolo_frame_skip": 3,
180              "yolo_tracker": "bytetrack.yaml",
181              "yolo_prefetch_workers": 2,
182              "min_action_window_duration": 2.0,
183              "ai_handoff_padding": 2.0,
184              "ai_max_workers": 5
185          },
186          "output": {
187              "default_highlights_name": "highlights.mp4",
188              "db_name": "sports_indexer.db"
189          },
190          "stitching": {
191              "begin_padding": 0.5,
192              "end_padding": 0.5,
193              "use_cache": false,
194              "min_shot_count": 1,

```

```

195         "watermark": {
196             "enabled": true,
197             "text": "Generated by Khelsutra.guru",
198             "position": "bottom-right",
199             "opacity": 0.85,
200             "logo_path": "",
201             "font_path": ""
202         }
203     }
204 }
205 ```
206
207     Notable knobs:
208     * `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for
noisy indoor-arena GoPro footage; see `docs/archives/TEST_RUN_SUMMARY.md` for why).
209     * `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
210     * `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
211     * `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower
values catch slower play, higher values are stricter about what counts as "action".
212     * `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30
fps ? 10 samples/sec.
213     * `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO
phase.
214     * `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate
window before sending to the AI provider.
215     * `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
216     * `stitching.min_shot_count` ? drop play segments whose estimated shot count is below
this threshold when compiling highlights. **Enforced on both the CLI runner and the
`/api/compile` endpoint** (segments that carry no `shot_count` metadata are exempt, so
explicitly-requested segment ids can't silently vanish under the default floor).
217     * `indexing.wasb.keep_frames` / `indexing.tracknet.keep_frames` ? after a
**successful** trajectory-detector run, the decoded frame cache and the staged video copy (the
dominant disk consumer ? ~GBs per 4K video) are deleted automatically. Default `false` (= auto-
delete); set `true` only for debugging. On **failure** they're always kept so a re-run resumes.
`indexing.{wasb,tracknet}.min_free_gb` warns before a run if WSL free space is below the
threshold (0 = off). See **$ Cache hygiene** below and `python -m tools.cleanup_caches`.
218     * `stitching.watermark` ? branding overlay burned into every highlight clip (**on by
default**). `enabled: false` turns it off; `text` is the fully-configurable string; `position` ?
{`bottom-right`, `bottom-left`, `top-right`, `top-left`}; `opacity` 0?1; optional `logo_path`
(transparent PNG). It ships with a **bundled Apache-2.0 font (Roboto Slab)** so the text renders
out-of-the-box **without a system fontconfig setup** (which Windows ffmpeg often lacks); set
**`font_path`** to a different `.ttf` (your own, or another in `backend/assets/fonts/`) to
change the typeface. If the watermark encode still fails, the clip is re-encoded without it (so
the reel is never lost) and the ffmpeg reason is logged.
219
220     ---
221
222     ## ? Frequently Asked Questions (FAQ)
223
224     ### Q1: The server complains that `ffprobe` is not found.
225     Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running
`scripts/doctor.py` outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a
PATH folder.
226
227     ### Q2: Why does my video fail the Relevance check?
228     Our default relevance check scans for the typical indoor green badminton court floor
color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust
the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and
`court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
229
230     ### Q3: How do I add a new custom validator to the pipeline?
231     1. Create a new python file in `backend/pipeline/validators/` (e.g. `my_check.py`).
232     2. Subclass `VideoValidator` and implement `validate(self, video_path, config,

```



```

context)``.
233     3. Register your class inside `backend/pipeline/validators/__init__.py` using
`registry.register(MyCheckValidator())`.
234     It's that simple! The app will automatically run it, display it in the CLI diagnostics
summary, and render it in the frontend checklist.
235
236     ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
237     * Print Buffering: Standard Python buffers console output when stdout is
redirected or running in background shells on Windows. To get real-time unbuffered log
statements immediately, run commands with Python's unbuffered flag:
238     ```bash
239     python -u -m backend.main --video-path "C:/path/to/video.mp4"
240     ```
241     * Video Seeking Overhead: Seeking to random frames in high-bitrate long H.265/HEVC
videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The
`motion` segmenter reads sequentially and skips frame decodes using container-level
`cap.grab()`, which delivers a 100x speedup on multi-gigabyte raw files. `yolo_hybrid`
sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.
242
243     ### Q5: Why did the video fail with "No keyframes / packet timestamps could be
extracted"?
244     Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes.
Standard Ffmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning
empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the
container demuxer level instead, which is incredibly robust and executes instantly without
decoding a single pixel.
245
246     ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB
video file?
247     You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API
payload. This restricts the segmenter frame loop to process only the first $N$ sub-sampled
frames, enabling you to test motion filters and court relevance checks in seconds:
248     ```bash
249     python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-
frames 300
250     ```
251     *(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60
seconds of video).*
252
253     ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
254     Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use
Google Gemini:
255     1. Create a local `.env` file in the project root (this is already in `.gitignore` to
prevent secret leaks).
256     2. Set your Google AI Studio API key in the file:
257     ```env
258     GEMINI_API_KEY=your_key_here
259     ```
260     3. Run with whichever segmenter you want:
261     ```bash
262     # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
263     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
264
265     # Full-video Gemini analysis (more expensive, highest semantic precision per call)
266     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
267
268     # Pure CV baseline ? no API key needed
269     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
270     ```
271     *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"/"motion"`) under
`"indexing"` in `config.json`.*
272
273     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers`.

```

274

275 ****Trajectory-based segmenters (`wasb\_hybrid`, `tracknet\_hybrid`).** The registry also includes two shuttle-trajectory variants. Like `yolo\_hybrid`, they cheaply filter the video down to candidate windows **locally** and then route each window to the AI provider (`gemini`) for precise boundaries ? so they need the same `GEMINI\_API\_KEY`. ****If an observability report shows `segmenter\_actual: wasb\_hybrid` (or `tracknet\_hybrid`), that is a valid, registered segmenter ? not an error.**** Pick one with `--segmenter wasb\_hybrid` or via `default\_segmenter`. (`wasb\_hybrid` additionally requires the WSL WASB setup ? see docs/TRACKNET_WSL_SETUP.md.)

276

277 **### Q8: How does the `yolo\_hybrid` pipeline actually save money?**

278 Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo\_hybrid` runs in four phases:

279

280 1. ****Chunking**** ? splits the video into overlapping windows (`chunk\_duration`, `chunk\_overlap`) so we can process incrementally.

281 2. ****Detection + tracking**** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds `yolo\_velocity\_threshold` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-`min\_action\_window\_duration` windows discarded.

282 3. ****AI handoff**** ? only the surviving candidate windows (padded by `ai\_handoff\_padding` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to `ai\_max\_workers`.

283 4. ****DB population**** ? confirmed segments are written to `play\_segments` along with `shot\_count`, `shot\_count\_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

284

285 In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.

286

287 **### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?**

288 Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native ****fast local trimming**** developer debugging feature:

289 1. Open your `config.json` file.

290 2. Under the `indexing` configuration block, add a `debug\_duration` key (in seconds):

```
291     ```json
292     "indexing": {
293         "default_segmenter": "gemini",
294         "debug_duration": 15.0,
295         ...
296     }
297     ```
```

298 3. When `debug\_duration` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

299

300 **### Q10: Can I trust the per-segment metadata from the `motion` segmenter?**

301 ****No.**** The zero-dependency `motion` segmenter detects **timing** via pixel-difference, but its per-segment ****metadata is heuristic/synthetic**** ? `intensity`, `avg\_speed\_kmh`, the serve/smash/foul/winner ****events****, and `server`/`receiver` are generated values, ****not measured****. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo\_hybrid` (default) or `gemini`. If your observability report shows a `synthetic\_motion\_metadata` warning, this is why.

302

303 **### Q11: I re-ran a video and my previous results changed/disappeared. Why?**

304 Videos are keyed by the ****MD5 of the file's bytes****. Re-processing the **same** file is treated as a fresh ingest: `add\_video` is `INSERT OR REPLACE`, and the foreign-key cascade ****deletes the previous play/candidate segments**** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest\_replaced\_prior` note in the report

flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

305

306 `### Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?`

307 Every processing run now writes a **per-video observability report** next to the video (or to `report.output_dir`` if set in `config.json``):

308 - ``.report.json`` ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.

309 - ``.report.md`` ? the **technical** report: validation results, processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ entry to read. **Start here when a run looks wrong.**

310 - ``.summary.md`` ? a **customer-friendly** summary: just the video metadata and highlights, no system internals.

311

312 To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a `? see README#Qn`` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}`` or `"report": {"output_dir": "...}"` in `config.json``.

313

314 `### Q13: A validation check failed and indexing was halted ? what do I do?`

315 Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY`` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_`` in `config.json``); **format** (not an MP4 container); **blur** (footage too soft ? improve sharpness/lighting or lower `validation.min_blur_threshold``; a reading of `0.0`` often means undecodable frames, so re-encode with `ffmpeg -i in.mp4 -c:v libx264 out.mp4`` before lowering the threshold); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json``, then re-run.

316

317 `### Q14: The report says fps was a "fallback" (not probed) ? what does that mean?`

318 The `Video`` section shows `FPS | 30 (fallback)`` and a `fps_fallback_used`` warning when the pipeline could **not probe** the real frame rate from the file and fell back to a hardcoded default (30 fps). Because rally detection converts frames/seconds and scales motion thresholds by fps, a wrong fps **miscalibrates timing and motion sensitivity** ? segment boundaries and which motion counts as "play" can be off.

319

320 This usually means validation didn't run (it's the step that probes the file via `ffprobe``), or `ffprobe`` couldn't read the stream's `r_frame_rate``. To fix: confirm the real fps ?

321 ````bash`

322 `ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of csv=p=0`

323 `"C:/path/to/video.mp4"`

324 `````

324 ? make sure `ffprobe`` is on your PATH (see [Q1](#q1-the-server-complains-that-ffprobe-is-not-found)) and that the run validates the file, then re-run so the real fps is used. (When fps shows `(probed)`` instead, this is all fine.)

325

326 `---`

327

328 `## ? Annotation & Evaluation`

329

330 Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)`` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

331

332 ````bash`

333 `python run_eval.py --scaffold --annotations rallies.csv # make an empty`

334 `template`

334 `# ...fill in start,end rows, then:`

335 `python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-`

336 `failures`

```

336     ``
337
338     It reports precision/recall/F1, mean IoU, and boundary error for two stages ? raw
339     detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result
340     is a detection problem or a segmentation problem.
341
342     > Rally timestamps evaluate and tune segmentation; they do not train the shuttle
343     detector (that needs per-frame coordinate labels). See
344     [docs/EVALUATION.md](docs/EVALUATION.md) for the annotation format, CLI flags, and how to
345     read the output.
346
347     ---
348
349     ## ? Running the Test Suite
350     The test suite uses `pytest` and lives in `tests/`. From the project root:
351
352     ```bash
353     pip install pytest pytest-cov --user
354
355     # One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
356     # runs the whole suite, reports a clear PASS/FAIL, and returns pytest's exit code:
357     python run_all_tests.py
358     python run_all_tests.py -k yolo -v          # any args pass straight through to pytest
359
360     # ...or call pytest directly:
361     pytest                                     # run everything
362     pytest tests/test_yolo_hybrid.py          # run a single file
363     pytest -k yolo --maxfail=1 -v             # filter by keyword, fail fast, verbose
364     pytest --cov=backend --cov-report=term-missing # coverage report (uses .coveragerc)
365     ```
366
367     > The optional `obsreport` package powers the per-video observability reports; its tests
368     > skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-
369     obsreport --no-build-isolation`.
370
371     The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
372     `VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
373
374     * `tests/test_base.py` ? segmenter registry
375     * `tests/test_database.py` ? SQLite schema, video/segment insert/query
376     * `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
377     * `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
378     * `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
379     and the Gemini provider
380     * `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
381     * `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
382     * `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
383     bailout, and timestamp-validation drops
384     * `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
385     runner (mocked subprocess) and the `tracknet_hybrid` segmenter
386     * `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
387     `tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
388     scoring, CLI)
389
390     ---
391
392     ## ? Project Structure (high level)
393
394     ``
395
396     backend/
397     main.py          # FastAPI app + CLI orchestrator
398     results.py       # Run-result aggregation
399     pipeline/        # The processing pipeline
400     segmenters/      # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
401     tracknet_hybrid, wasb_hybrid, fusion_hybrid

```

```

388     detectors/           # Shuttle/person detectors: tracknet, wasb, person
389     validators/         # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
390     stitcher/compiler.py # FFmpeg-based highlight compiler
391     api/static/          # Zero-build frontend served by FastAPI
392     infrastructure/      # SQLite layer (database.py) + execution policy
393     providers/           # Pluggable AI providers (gemini today)
394     sports/              # Pluggable sport profiles (badminton today)
395     eval/                # Rally evaluation harness: metrics, annotations, scoring
396     annotations/         # Rally annotation providers
397     config/              # Config schema + model registry
398     features/            # Rally sequence features
399     reporting/           # Observability run signals
400     tools/               # Export + rally slicing utilities
401     utils/               # ffmpeg/ffprobe + bbox geometry helpers
402     docs/                # Live docs (CODE_MAP, COMMERCIALIZATION, BACKLOG, ROADMAP,
guidelines, golden-set). Historical items in docs/archives/
403     tests/               # pytest suite (mocked, runs offline)
404     scratch/             # Throwaway experiments
405     training/            # Offline detector training (see ROADMAP)
406     eval_baselines/      # Frozen evaluation baselines
407     scripts/
408     doctor.py            # Dependency & FFmpeg diagnostics + default config init
409     run_process_and_stitch.py # Interactive end-to-end runner
410     tools/
411     cleanup_caches.py    # Dry-run-first cache GC (WSL ~/clips + output/); see "Cache
Hygiene" above
412     run_all_tests.py     # Full test-suite wrapper (see "Running the Test Suite" above)
413     run_eval.py          # Rally evaluation CLI (see "Annotation & Evaluation" above)
414     run_regression.py     # Regression runner
415     pyproject.toml       # PEP 621 packaging (hatchling); `indexer` entry point + deps
416     config.json          # Runtime configuration (see "Threshold Tuning" above)
417     ``
418

```

9. assistant (2026-06-21T13:35:01.680Z)

Now let me look at the config loading, requirements.txt, scripts directory, and the doctor.py setup script.

10. user (2026-06-21T13:35:02.661Z)

```

1     fastapi>=0.111.0
2     uvicorn>=0.30.1
3     opencv-python>=4.9.0.80
4     numpy>=1.26.4
5     pydantic>=2.7.1
6     jinja2>=3.1.4
7     python-dotenv>=1.0.0
8     google-genai>=2.4.0
9     torch>=2.12.0
10    torchvision>=0.27.0
11    supervision>=0.21.0,<0.30.0
12
13    # Per-video observability reports (SOFT dependency: the pipeline degrades to a
14    # no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
15    # the private repo is published, e.g.:
16    # obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
17    # For local development install it editable from a sibling checkout:
18    # pip install -e ../sports-obsreport --no-build-isolation
19

```

11. user (2026-06-21T13:35:03.190Z)

```
scripts\README.md
scripts\nightly_regression.ps1
scripts\register_nightly_task.ps1
scripts\nightly_regression.py
scripts\run_phase3_eval.py
scripts\doctor.py
scripts\run_process_and_stitch.py
```

12. user (2026-06-21T13:35:03.203Z)

```
backend\config\loader.py
backend\config\presets.py
backend\config\__init__.py
backend\config\startup.py
backend\config\models.py
```

13. user (2026-06-21T13:35:06.279Z)

```
1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from .models import AppConfig
20     from .presets import (
21         COMPUTE_TARGET_FOR_PROFILE,
22         PROFILE_PRESETS,
23         is_known_profile,
24         preset_for,
25         suggest_profile,
26     )
27
28     logger = logging.getLogger(__name__)
29
30     DEFAULT_CONFIG_PATH = Path("config.json")
31
32
33     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
34         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
35         scalars
36         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
37         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
38         profile
39         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
40         out = dict(base)
41         for key, val in override.items():
42             if isinstance(val, dict) and isinstance(out.get(key), dict):
43                 out[key] = _deep_merge(out[key], val)
44             else:
45                 out[key] = val
46         return out
```

```

46
47     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
48         """The EXPLICITLY chosen deployment profile, env winning over config.json. ```` if
49         none
50         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
51         a bare
52         environment never silently selects a profile (and never silently spends on cloud).
53         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
54         file's
55         profile (rather than suppressing it) ? so an env typo can't leave the recorded
56         profile
57         asserting a preset that was never applied. A bad value in config.json is left to
58         surface as
59         a hard, typed-Literal error downstream (config-file correctness)."""
60         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
61         if env_profile and env_profile.strip():
62             ep = env_profile.strip()
63             if is_known_profile(ep):
64                 return ep
65             logger.warning(
66                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
67                 it "
68                 "and falling back to config.json deployment.profile.", ep, ",
69             ".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
72             choice)
73         dep = file_data.get("deployment")
74         if isinstance(dep, dict):
75             prof = dep.get("profile")
76             if isinstance(prof, str) and prof.strip():
77                 return prof.strip()
78         return ""
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
81     "") -> list[str]:
82         """Dotted paths in `data` that the typed config will not honor.
83
84         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
85         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
86         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
87         user's intent is lost ? collect every such key so load_config can warn.
88         """
89         unknown: list[str] = []
90         fields = model_cls.model_fields
91         for key, val in data.items():
92             if key not in fields:
93                 unknown.append(prefix + key)
94                 continue
95             ann = fields[key].annotation
96             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
97             BaseModel):
98                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
99         return unknown
100
101     def _get_builtin_defaults() -> dict[str, Any]:
102         """Return a plain dict of the current built-in defaults.
103
104         The AppConfig model is the single source of truth for defaults.
105         """
106         return AppConfig().model_dump(mode="json")

```

```

101
102 def load_config(path: str | Path | None = None) -> AppConfig:
103     """Load configuration with the following precedence:
104
105     1. Built-in defaults defined in the Pydantic models.
106     2. Values from the JSON file (if present and readable).
107     3. (Future) environment / CLI overrides.
108
109     Missing file or unreadable file ? returns a fully defaulted AppConfig.
110     Unknown keys in the file are tolerated (extra="allow" on the model).
111     """
112     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
113
114     file_data: dict[str, Any] = {}
115     if cfg_path.exists():
116         try:
117             with open(cfg_path, "r", encoding="utf-8") as f:
118                 file_data = json.load(f)
119         except Exception as exc:
120             # Non-fatal: continue with defaults + whatever we could parse.
121             # In production we might log here.
122             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
123 data.")
124     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
125 `null`) would
126     # otherwise crash startup: a truthy non-mapping hits `.items()` in
127 _unknown_key_paths, and any
128     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
129 warn, per the
130     # loader's "bad input is non-fatal" contract.
131     if not isinstance(file_data, dict):
132         logger.warning("%s is not a JSON object (got %s); ignoring it and using
133 defaults.",
134                        cfg_path, type(file_data).__name__)
135     file_data = {}
136
137 if file_data:
138     unknown = _unknown_key_paths(file_data, AppConfig)
139     if unknown:
140         logger.warning(
141             "[CONFIG WARNING] %s contains keys the typed config does not "
142             "recognize (typo'd or removed knobs ? top-level extras are kept "
143             "but unread; unknown section keys are silently dropped): %s",
144             cfg_path, ", ".join(sorted(unknown)),
145         )
146
147 # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
148 at
149 # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
150 ONLY when
151 # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
152 With no
153 # profile the merge is byte-identical to the pre-M1 loader, so the default path is
154 unchanged.
155 defaults = _get_builtin_defaults()
156 profile = _resolve_explicit_profile(file_data)
157
158 if profile and not is_known_profile(profile):
159     # Only an unrecognised value from config.json reaches here (env typos already
160 fell back
161     # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
162 the
163     # typed Literal at model_validate below, a hard clear error ? loud, never
164 silent.

```



```

154         logger.warning(
155             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
156             profile, ", ".join(PROFILE_PRESETS),
157         )
158         profile = ""
159
160     if profile:
161         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
162         # Record the profile actually applied ? the env may have chosen it over
config.json,
163         # so re-stamp it onto the merged deployment section to keep the record honest.
164         merged.setdefault("deployment", {})
165         if isinstance(merged["deployment"], dict):
166             merged["deployment"]["profile"] = profile
167             ct = merged["deployment"].get("compute_target")
168             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
169             if not ct and canonical:
170                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
171                 # to "") gets its canonical target back ? the record never silently lies
about an
172                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
173                 merged["deployment"]["compute_target"] = canonical
174                 elif ct and canonical and ct != canonical:
175                     logger.warning(
176                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
177                         "config.json overrides it to %r ? honoring the explicit override.",
178                         profile, canonical, ct,
179                     )
180                     logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
181                                 profile, merged["deployment"].get("compute_target"))
182     else:
183         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
184         merged = {**defaults, **file_data}
185         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
186         if suggestion:
187             logger.debug(
188                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
189                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
190                 "auto-applies ? D15).", suggestion,
191             )
192
193     return AppConfig.model_validate(merged)
194
195
196 def save_default_config(path: str | Path | None = None) -> Path:
197     """Write a fresh default config.json.
198
199     Used by scripts/doctor.py diagnostics and the --setup CLI path.
200     Overwrites if the file exists.
201     """
202     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
203     cfg = AppConfig()
204     cfg_path.parent.mkdir(parents=True, exist_ok=True)
205     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
206     return cfg_path
207
208

```

```

209 # Backwards-compatible alias used by some older call sites during transition.
210 def get_default_config() -> dict[str, Any]:
211     """Return defaults as a plain dict (for transitional use only)."""
212     return _get_builtin_defaults()
213

```

14. user (2026-06-21T13:35:06.587Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil
5
6     # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7     # root (requirements.txt, config.json). Resolve the root from this file's
8     # location so it works regardless of the caller's cwd.
9     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11     def check_command(cmd):
12         """Check if a system command is available."""
13         return shutil.which(cmd) is not None
14
15     def run_pip_install():
16         """Install required Python packages."""
17         print("Installing Python packages from requirements.txt...")
18
19         # Check if NVIDIA GPU is present
20         has_gpu = check_command("nvidia-smi")
21
22         requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25         import platform
26         if has_gpu:
27             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
28             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
29         elif platform.system() == "Darwin":
30             # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
31             print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
32         else:
33             print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
34             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
35
36         try:
37             subprocess.check_call(pip_cmd)
38             print("[SUCCESS] Python packages installed.")
39             return True
40         except subprocess.CalledProcessError as e:
41             print(f"[ERROR] Failed to install Python packages: {e}")
42             return False
43
44     def init_default_config():
45         """Initialize a default config.json if it doesn't exist.
46
47         Now delegates to the canonical backend.config loader so that defaults
48         stay in sync with the Pydantic models (single source of truth).
49         """
50         try:
51             from backend.config import save_default_config
52         except Exception as e:
53             print(f"[ERROR] Could not import backend.config loader: {e}")
54             print("[INFO] Falling back to no-op for config initialization.")
55             return

```

```

56
57     config_path = os.path.join(REPO_ROOT, "config.json")
58     if os.path.exists(config_path):
59         print(f"[INFO] {config_path} already exists. Skipping initialization.")
60         return
61
62     try:
63         saved_path = save_default_config(config_path)
64         print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
65     except Exception as e:
66         print(f"[ERROR] Failed to create config.json via loader: {e}")
67
68     def run_diagnostics():
69         """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
70         print("-" * 50)
71         print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
72         print("-" * 50)
73
74         # 1. Check Python Version
75         print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
76         if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
77             print("[WARNING] Python 3.8+ is highly recommended!")
78         else:
79             print("[PASS] Python version check passed.")
80
81         print("-" * 50)
82
83         # 2. Check FFmpeg / FFprobe
84         ffmpeg_ok = check_command("ffmpeg")
85         ffprobe_ok = check_command("ffprobe")
86
87         if ffmpeg_ok:
88             print("[PASS] FFmpeg is installed and available in PATH.")
89         else:
90             print("[FAIL] FFmpeg is NOT found in your PATH.")
91
92         if ffprobe_ok:
93             print("[PASS] FFprobe is installed and available in PATH.")
94         else:
95             print("[FAIL] FFprobe is NOT found in your PATH.")
96
97         if not (ffmpeg_ok and ffprobe_ok):
98             import platform
99             print("\n" + "!" * 50)
100             print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
101             _sys = platform.system()
102             if _sys == "Windows":
103                 print("Install on Windows: winget install Gyan.FFmpeg")
104             elif _sys == "Darwin":
105                 print("Install on macOS: brew install ffmpeg")
106             else:
107                 print("Install on Linux: sudo apt-get install -y ffmpeg (or your
distro's package manager)")
108             print("Or download a build from https://ffmpeg.org/download.html and put
ffmpeg/ffprobe on PATH.")
109             print("!" * 50 + "\n")
110         else:
111             print("[SUCCESS] All essential media command line tools are ready.")
112
113         print("-" * 50)
114
115         # 3. Check ML / GPU (Optional but good)

```

```

116         try:
117             import torch
118             cuda_ok = torch.cuda.is_available()
119             if cuda_ok:
120                 print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
121             else:
122                 print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
123         except ImportError:
124             print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
125
126         print("=" * 50)
127         return ffmpeg_ok and ffprobe_ok
128
129     if __name__ == "__main__":
130         init_default_config()
131         media_ok = run_diagnostics()
132         run_pip_install()
133         print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
134

```

15. assistant (2026-06-21T13:35:09.950Z)

Now let me check the OutputConfig defaults (db_name, output_dir), the StorageConfig, and where scratch dirs are resolved. Let me also check the scripts README and how ``.env`/`config.json`` are discovered relative to cwd.

16. user (2026-06-21T13:35:11.315Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):

```

```

34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
74         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
75         device: str = "cuda" # auto | cuda | mps | cpu
76         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
77         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
78         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
80         keep_frames: bool = False
81         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
82         min_free_gb: float = 0.0
83         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
84         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
85         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
86         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
87         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).

```

```

87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                               behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)
119        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120        court_polygon: Optional[list[list[float]]] = None
121        # Net line for the person two-sided-motion split (person features only ? the shuttle
122        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123        net_axis: Literal["x", "y"] = "y"
124        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0

```

```

141     detector_model: str = "fasterrcnn_resnet50_fpn"
142     detector_score_threshold: float = 0.5
143     yolo_velocity_threshold: float = 0.15
144     yolo_frame_skip: int = 3
145     yolo_tracker: str = "bytetrack.yaml"
146     yolo_prefetch_workers: int = 2
147     min_action_window_duration: float = 2.0
148     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155     max_window_duration: float = 6.0
156     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160     # Only cuts (recall can't drop). OFF by default until measured/promoted.
161     window_hard_cap: bool = False
162     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164     # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
168     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169     window_inactivity_end: float = 1.5
170     inactivity_rally_thresh: float = 0.20
171     inactivity_min_density: float = 0.30
172     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180     window_blob_fallback: bool = False

```

```

181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
218         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220             # step makes `while t < duration: t += step` loop forever, growing memory before
any
221             # GPU work. Reject the bad config at load time instead.
222             if self.chunk_overlap >= self.chunk_duration:
223                 raise ValueError(
224                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                     f"({self.chunk_duration}); otherwise chunking never advances."
226                 )
227             return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).

```



```

235     output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process_video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).

```

```

292     max_upload_mb: int = 4096
293     max_part_mb: int = 95
294     allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296     # --- M9 edge auth (Cloudflare Access, D9) ---
297     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302     # application's audience tag. NO secrets here ? these are public identifiers.
303     edge_auth_enabled: bool = False
304     cf_team_domain: str = ""          # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305     cf_access_aud: str = ""          # the CF Access application AUD tag the token must
carry
306     # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the
hosted
307     # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not
a cookie.)
308     cors_allow_origins: List[str] = ["*"]
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
313         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
314         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
315         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
316         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
317         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
318         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
319         output_root: str = ""
320         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
321         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
322         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
323         # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
324         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
325         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
326         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
327         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
328         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
329         input_root: str = ""
330         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
331         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
332         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root

```

```

334 # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335 # the legacy flat layout is used unchanged.
336 single_folder_per_video: bool = False
337 # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338 workspace_root: str = ""
339 # Cloud namespace prefix under the root so per-video folders sit tidily beside
340 # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341 workspace_prefix: str = "workspaces"
342 local: dict[str, Any] = Field(default_factory=dict)
343 s3: dict[str, Any] = Field(default_factory=dict)
344 gcs: dict[str, Any] = Field(default_factory=dict)
345 gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348 class DeploymentConfig(BaseModel):
349     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
351
352     **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353     behaves exactly as before this section existed. A profile is opt-in (``config.json``
354     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368 class BillingConfig(BaseModel):
369     """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371     **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372     today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373     ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374     version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375     inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377     ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378     wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379     pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381     enabled: bool = False
382     pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to

```

```

a real one
383         fx_inr_per_usd: float = 83.0                # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0                        # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
394         sport: Sport = "badminton"
395         ai_provider: str = "gemini"
396         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
397         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398         ai_model: str = "gemini-flash-latest"
399
400         validation: ValidationConfig = Field(default_factory=ValidationConfig)
401         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
402         output: OutputConfig = Field(default_factory=OutputConfig)
403         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404         security: SecurityConfig = Field(default_factory=SecurityConfig)
405         logging: LoggingConfig = Field(default_factory=LoggingConfig)
406         storage: StorageConfig = Field(default_factory=StorageConfig)
407         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408         billing: BillingConfig = Field(default_factory=BillingConfig)
409
410         # Allow extra keys during transition so existing config.json files do not break.
411         model_config = {
412             "extra": "allow",
413             "validate_assignment": True,
414         }
415
416         # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
417         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
418         # attribute access / the typed config. A nested *section* is returned as a plain
dict
419         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420         # section model instead breaks it (a model has no .get), which bit every validator
on
421         # real runs. Scalar leaves are returned as-is.
422         def get(self, key: str, default: Any = None) -> Any:
423             _MISSING = object()
424             val: Any = _MISSING
425             if key in type(self).model_fields or hasattr(self, key):
426                 val = getattr(self, key, _MISSING)
427             if val is _MISSING:
428                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
429                 # discovered dynamically from the model fields, so a future top-level
section
430                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431                 for fname in type(self).model_fields:
432                     section = getattr(self, fname, None)
433                     if isinstance(section, BaseModel) and hasattr(section, key):
434                         val = getattr(section, key)
435                         break
436             if val is _MISSING:

```

```

437         return default
438     if isinstance(val, BaseModel):
439         return val.model_dump(mode="json")
440     return val
441
442     def __getitem__(self, key: str) -> Any:
443         val = self.get(key)
444         if val is None and key not in type(self).model_fields:
445             # Re-raise like a real dict for unknown top-level
446             raise KeyError(key)
447         return val
448
449
450     # Convenience type alias for callers that want to annotate
451     Config = AppConfig
452

```

17. assistant (2026-06-21T13:35:14.915Z)

Now I have a clear picture. Let me verify a couple of remaining items: the scripts README, how ``.env`` is discovered (load_dotenv with no arg = cwd-relative), and whether there is a ``.backend/api/static`` directory with content. Let me also check the presets for the profiles.

18. user (2026-06-21T13:35:15.885Z)

```

1     # scripts/
2
3     **Standalone run-drivers / one-off entry CLIs** ? scripts with a `__main__` that you
*run*, not
4     *import*. Run as `python scripts/<name>.py`.
5
6     What belongs here (vs. elsewhere ? see `docs/CODE_MAP.md` §0 for the full table):
7     - ? a CLI that orchestrates a pipeline/eval run end-to-end (e.g. `run_phase3_eval.py`,
`run_process_and_stitch.py`)
8     - ? **importable eval library** (metrics, calibration, a scorer) ? `backend/eval/`
9     - ? an **ops/maintenance tool** run as `python -m tools.x` ? `tools/`
10    - ? a **throwaway repro / debug** script ? `scratch/` (gitignored)
11    - ? anything imported by app code ? it's library, put it under `backend/`
12
13    If a script needs `load_dotenv()` / `sys.path` setup before importing `backend`, add
14    `"scripts/<name>.py" = ["E402"]` to `ruff.toml` (import-order is load-bearing there).
15
16    Grandfathered at the repo root (referenced by CI/packaging/tests ? left in place):
17    `run_all_tests.py`, `run_regression.py`, `run_eval.py`. **New** run-drivers go here, not
the root.
18
19    `doctor.py` here is the diagnostics/bootstrap script (FFmpeg + CUDA checks, default-
config
20    init, `pip install -r requirements.txt`). It was formerly the root `setup.py`; it was
renamed
21    and moved here so it no longer shadows the PEP 517 build system now that
`pyproject.toml` exists.
22    Run it as `python scripts/doctor.py` or `indexer --setup`.
23

```

19. user (2026-06-21T13:35:16.344Z)

```

1     """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3     A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4     config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5     knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6

```

```

7     Precedence (lowest ? highest), realised by the loader::
8
9     built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is pure data + pure functions ? no IO, and (critically) no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (`config.json` `deployment.profile` or the `DEPLOYMENT_PROFILE` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (`profile=""` ==
15     today). Auto-detect only ever suggests a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     ""
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. `models.DeploymentConfig`'s `Literal`
mirrors
26     # these and `tests/test_config_deployment.py` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
`deployment.{profile,compute_target}` plus
40     # `indexing.{detector_impl,skip_ai_handoff}` and `storage.backend`. Later milestones
extend
41     # these bundles ? `input_backend` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55             "storage": {"backend": "gcs"},
56         },
57         "cpu-mock": {
58             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},

```

```

61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()
107
108
109 def suggest_profile(
110     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111 ) -> str:
112     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114     Signals, in priority order:
115
116     - ``K_SERVICE`` set (Cloud Run)          ? ``"cloud-serving"``

```

```

117         - ``CI`` truthy ? ``"cpu-mock"``
118         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119         - otherwise ? ``""`` (no strong suggestion)
120
121     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124     """
125     e = os.environ if env is None else env
126     if e.get("K_SERVICE"):
127         return "cloud-serving"
128     if _truthy(e.get("CI")):
129         return "cpu-mock"
130     if probe_cuda and _cuda_available():
131         return "truly-local"
132     return ""
133

```

20. user (2026-06-21T13:35:16.886Z)

```

backend\api\static\index.html
backend\api\static\style.css
backend\api\static\app.js

```

21. assistant (2026-06-21T13:35:21.265Z)

The `backend/api/static` directory has content (index.html, style.css, app.js) ? not entirely dead. Let me verify how `.env` and config.json discovery is cwd-relative by confirming `load\_dotenv()` is called with no path (line 10 of main.py confirmed) and check if the static dir is the legacy zero-build UI. Let me also confirm the worker's relationship to the server (separate process) and check the nightly/process scripts briefly.

22. user (2026-06-21T13:35:22.163Z)

```

1      <!DOCTYPE html>
2      <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <meta name="viewport" content="width=device-width, initial-scale=1.0">
6          <title>Badminton Highlight & Rally Indexer</title>
7          <link rel="stylesheet" href="/static/style.css">
8      </head>
9      <body>
10         <div class="app-container">
11             <!-- HEADER -->
12             <header>
13                 <div class="logo-section">
14                     <h1 id="app-title">Badminton Highlight & Rally Indexer</h1>
15                     <p>Local AI-powered video validation, segmentation, and clip
indexing</p>
16                 </div>
17                 <div class="status-badge">
18                     <span class="status-indicator"></span>
19                     <span>Local Server Active</span>
20                 </div>
21             </header>
22
23             <!-- MAIN GRID LAYOUT -->
24             <div class="main-grid">
25                 <!-- SIDEBAR: Controls & Validation -->
26                 <div class="sidebar">

```



```

27         <!-- Video File Input Panel -->
28         <div class="glass-panel" id="video-input-panel">
29             <h2 class="panel-title">
30                 <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path
d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0 1-2 2V8a2 2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"/><circle
cx="12" cy="13" r="4"/></svg>

```

23. user (2026-06-21T13:35:23.710Z)

```

backend\main.py:8:from dotenv import load_dotenv
backend\main.py:10:load_dotenv()
backend\eval\gemini_refine.py:17:Run (GEMINI_API_KEY in env):
backend\eval\gemini_refine.py:105:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:107:         raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:156:     api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:158:         raise SystemExit("GEMINI_API_KEY not set in env")

```

24. assistant (2026-06-21T13:35:26.673Z)

I have all the information I need. Let me verify one more thing ? how the Gemini provider reads the API key (to confirm it's env-based via the cwd-relative `.env`).

25. user (2026-06-21T13:35:27.770Z)

```

backend\providers\__init__.py:14:     def get(self, name: str, api_key: str, model_name: str) ->
AIVideoProvider:
backend\providers\__init__.py:17:         return self._registry[name](api_key=api_key,
model_name=model_name)
backend\providers\base.py:5:     def __init__(self, api_key: str, model_name: str):
backend\providers\base.py:6:         self.api_key = api_key
backend\providers\gemini.py:20:     def __init__(self, api_key: str, model_name: str):
backend\providers\gemini.py:21:         super().__init__(api_key, model_name)
backend\providers\gemini.py:28:         self._client = genai.Client(api_key=self.api_key)

```

26. user (2026-06-21T13:35:31.650Z)

```

backend\pipeline\segmenters\gemini.py:90:         api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:80:         api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:77:         impl =
(os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
backend\pipeline\segmenters\trajectory_hybrid.py:169:         api_key =
os.environ.get(api_key_env_var)

```

27. assistant (2026-06-21T13:35:34.523Z)

I now have a complete picture of the launch and packaging surface. Let me compile my findings.

28. user (2026-06-21T13:36:46.945Z)

Structured output provided successfully

29. assistant (2026-06-21T13:36:54.091Z)

I've completed the analysis of the app launch and packaging surface. The structured findings are returned above. Key takeaways:

- ****One entrypoint, one process for serving****: `indexer = backend.main:main` is the sole console script; `--server` runs a self-contained FastAPI server with an in-process ThreadPool job worker. `python -m backend.worker` is a separate storage-aware orchestrator for the cloud/decoupled-compute path, not a daemon the server talks to.
- ****`pip install .` is incomplete by design****: it omits torch/torchvision (out-of-band `--index-url`), and ffmpeg/ffprobe are unmanaged system deps ? so a bare wheel can't run the GPU/trajectory paths.

- **All state is CWD-relative**: config.json, .env, the SQLite DB (under `output/`), output dir, and the static UI mount all anchor on the launch directory with no app-home concept ? the central thing a packaging layer must fix.
- **A drift already exists**: `requirements.txt` lists torch while `pyproject.toml` deliberately omits it, and `doctor.py` writes config to REPO_ROOT while the server reads it from CWD.